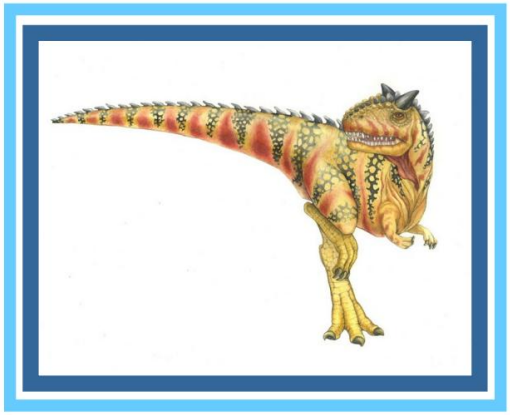
Machine Translated by Google

Capítulo 2: Sistema operativo Servicios



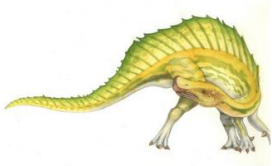
Silberschatz, Galvin y Gagne ©2018

Machine Translated by Google



Describir

- Servicios del sistema operativo
- Interfaz de usuario y del sistema operativo
- Llamadas al sistema
- Servicios del sistema
- Enlazadores y cargadores
- Por qué las aplicaciones son específicas del sistema operativo
- Diseño e implementación
- Estructura del sistema operativo
- Creación e inicio de un sistema operativo
- Depuración del sistema operativo



Machine Translated by Google



Objetivos

Identificar los servicios que proporciona un sistema operativo.

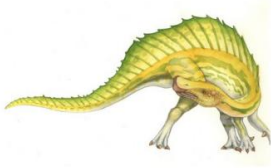
Ilustrar cómo se utilizan las llamadas al sistema para proporcionar servicios del sistema operativo.

Comparar y contrastar las estrategias monolíticas, por capas, de microkernel, modulares e híbridas para el diseño de sistemas operativos.

Ilustre el proceso de arranque de un sistema operativo.

Aplicar herramientas para supervisar el rendimiento del sistema operativo

Diseñar e implementar módulos del núcleo para interactuar con un núcleo de Linux



Machine Translated by Google



Servicios del sistema operativo

Los sistemas operativos proporcionan un entorno para la ejecución de programas.

y servicios a programas y usuarios

Un conjunto de servicios del sistema operativo proporciona funciones que resultan útiles para el usuario:

- Interfaz de usuario : casi todos los sistemas operativos tienen una interfaz de usuario (UI).

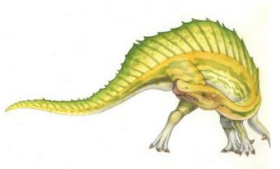
  Varía entre la línea de comandos (CLI) y el usuario de gráficos.
  Interfaz gráfica de usuario (GUI), pantalla táctil, lote

- Ejecución del programa : El sistema debe ser capaz de cargar un programa en la memoria y ejecutarlo, finalizando la ejecución, ya sea de forma normal o anormal (indicando un error).

- Operaciones de E/S : Un programa en ejecución puede requerir E/S, lo que puede implicar un archivo o un dispositivo de E/S.

- Manipulación del sistema de archivos : el sistema de archivos es de particular importancia.

  Interés. Los programas necesitan leer y escribir archivos y directorios, crearlos y eliminarlos, buscarlos, listar información de archivos, administrar permisos.



Machine Translated by Google



Servicios del sistema operativo (continuación)

Un conjunto de servicios del sistema operativo proporciona funciones que son útiles para el usuario (Continuación):

- Comunicaciones : Los procesos pueden intercambiar información, en el mismo ordenador o entre ordenadores a través de una red.

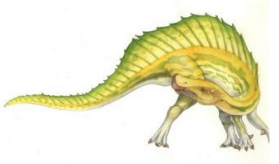
  Las comunicaciones pueden realizarse a través de memoria compartida o mediante paso de mensajes (paquetes movidos por el sistema operativo).

- Detección de errores : el sistema operativo debe estar constantemente al tanto de posibles errores. errores

  Puede ocurrir en el hardware de la CPU y la memoria, en los dispositivos de E/S, en el programa del usuario.

  Para cada tipo de error, el sistema operativo debe tomar las medidas adecuadas para garantizar un funcionamiento correcto y coherente.

  Las herramientas de depuración pueden mejorar enormemente la capacidad del usuario y del programador para utilizar el sistema de manera eficiente.



Machine Translated by Google



Servicios del sistema operativo (continuación)

Existe otro conjunto de funciones del sistema operativo para garantizar el funcionamiento eficiente de el propio sistema mediante el intercambio de recursos

- Asignación de recursos: cuando hay varios usuarios o varios trabajos
  Al ejecutarse simultáneamente, se deben asignar recursos a cada uno de ellos.

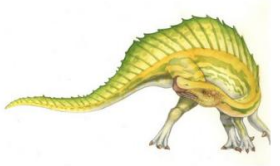
  Muchos tipos de recursos: ciclos de CPU, memoria principal, archivos
  Almacenamiento, dispositivos de E/S.

- Registro: Para llevar un registro de qué usuarios usan cuántos y qué tipos de recursos informáticos.

- Protección y seguridad: los propietarios de la información almacenada en un
  En sistemas informáticos multiusuario o en red, es posible que se desee controlar el uso de esa información; los procesos concurrentes no deben interferir entre sí.

  La protección implica garantizar que todo acceso a los recursos del sistema esté controlado.

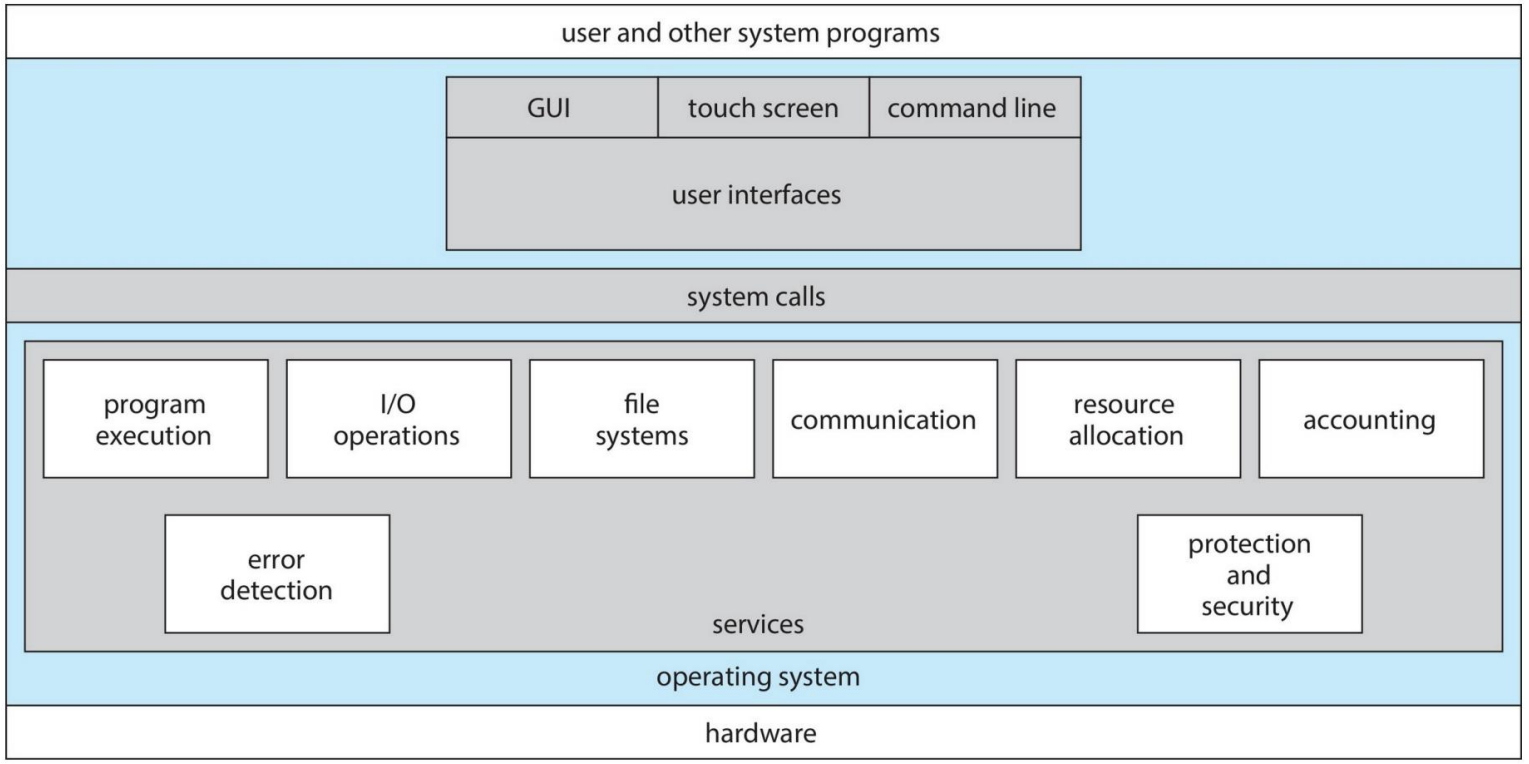
  La seguridad del sistema frente a personas externas requiere del usuario
  La autenticación se extiende a la protección de los dispositivos de E/S externos contra intentos de acceso no válidos.

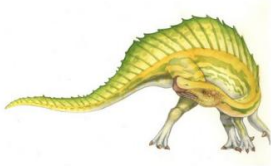


Machine Translated by Google



Una visión de los servicios del sistema operativo





Machine Translated by Google



Intérprete de línea de comandos

La interfaz de línea de comandos (CLI) permite la entrada directa de comandos.

A veces se implementa en el núcleo, a veces mediante un programa del sistema.

A veces se implementan varios sabores: cáscaras

Principalmente obtiene un comando del usuario y lo ejecuta.

A veces son comandos integrados, a veces solo nombres de programas.

- Si se trata de lo último, agregar nuevas funciones no requiere modificar el shell.

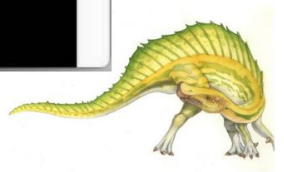


Machine Translated by Google



Intérprete de comandos de Bourne Shell

```
1. root@r6181-d5-us01:~ (ssh)
root@r6181-d5-u... ⌘1   ssh ⌘2   root@r6181-d5-us01... ⌘3
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem            Size  Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
                       50G   19G   28G  41% /
tmpfs                 127G  520K  127G   1% /dev/shm
/dev/sda1             477M   71M  381M  16% /boot
/dev/dssd0000         1.0T  480G  545G  47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs
                       12T  5.7T  6.4T  47% /mnt/orangefs
/dev/gpfs-test         23T  1.1T   22T   5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2  6.6 42665344 17520636 ?   S<Ll Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root      69849  6.6  0.0      0     0 ?        S    Jul12 181:54 [vpthread-1-1]
root      69850  6.4  0.0      0     0 ?        S    Jul12 177:42 [vpthread-1-2]
root       3829  3.0  0.0      0     0 ?        S    Jun27 730:04 [rp_thread 7:0]
root       3826  3.0  0.0      0     0 ?        S    Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x------ 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```



Machine Translated by Google



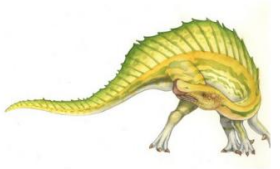
Interfaz de usuario del sistema operativo (GUI)

Interfaz de escritorio fácil de usar

- Normalmente ratón, teclado y monitor
- Los iconos representan archivos, programas, acciones, etc.
- Varios botones del ratón sobre los objetos de la interfaz provocan diversas acciones (proporcionar información, opciones, ejecutar función, abrir directorio (conocido como carpeta)).
- Inventado en Xerox PARC

Muchos sistemas ahora incluyen interfaces tanto de línea de comandos (CLI) como de interfaz gráfica de usuario (GUI).

- Microsoft Windows es una interfaz gráfica de usuario (GUI) con una consola de comandos (CLI).
- Apple Mac OS X utiliza la interfaz gráfica de usuario “Aqua” con núcleo UNIX. debajo y conchas disponibles
- Unix y Linux tienen CLI con interfaces GUI opcionales (CDE, KDE, GNOMO)



Machine Translated by Google

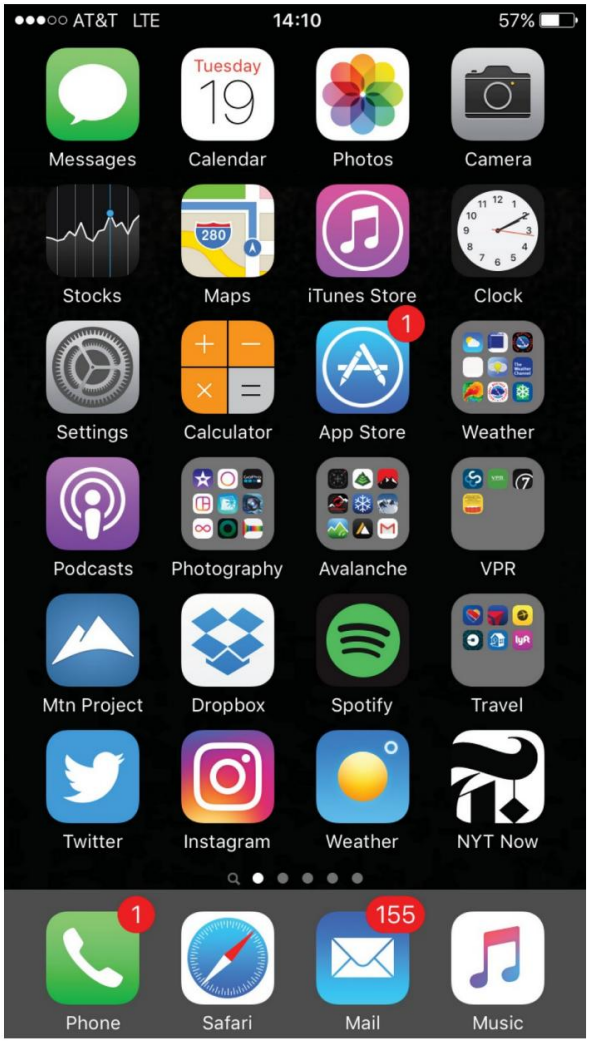


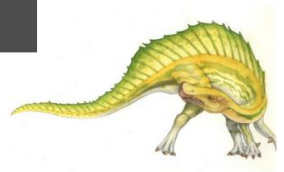
Interfaces de pantalla táctil

Los dispositivos con pantalla táctil requieren nuevas interfaces.

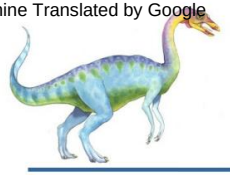
- No es posible o no se desea usar el ratón.
- Acciones y selección basadas en gestos
- Teclado virtual para la introducción de texto

Comandos de voz

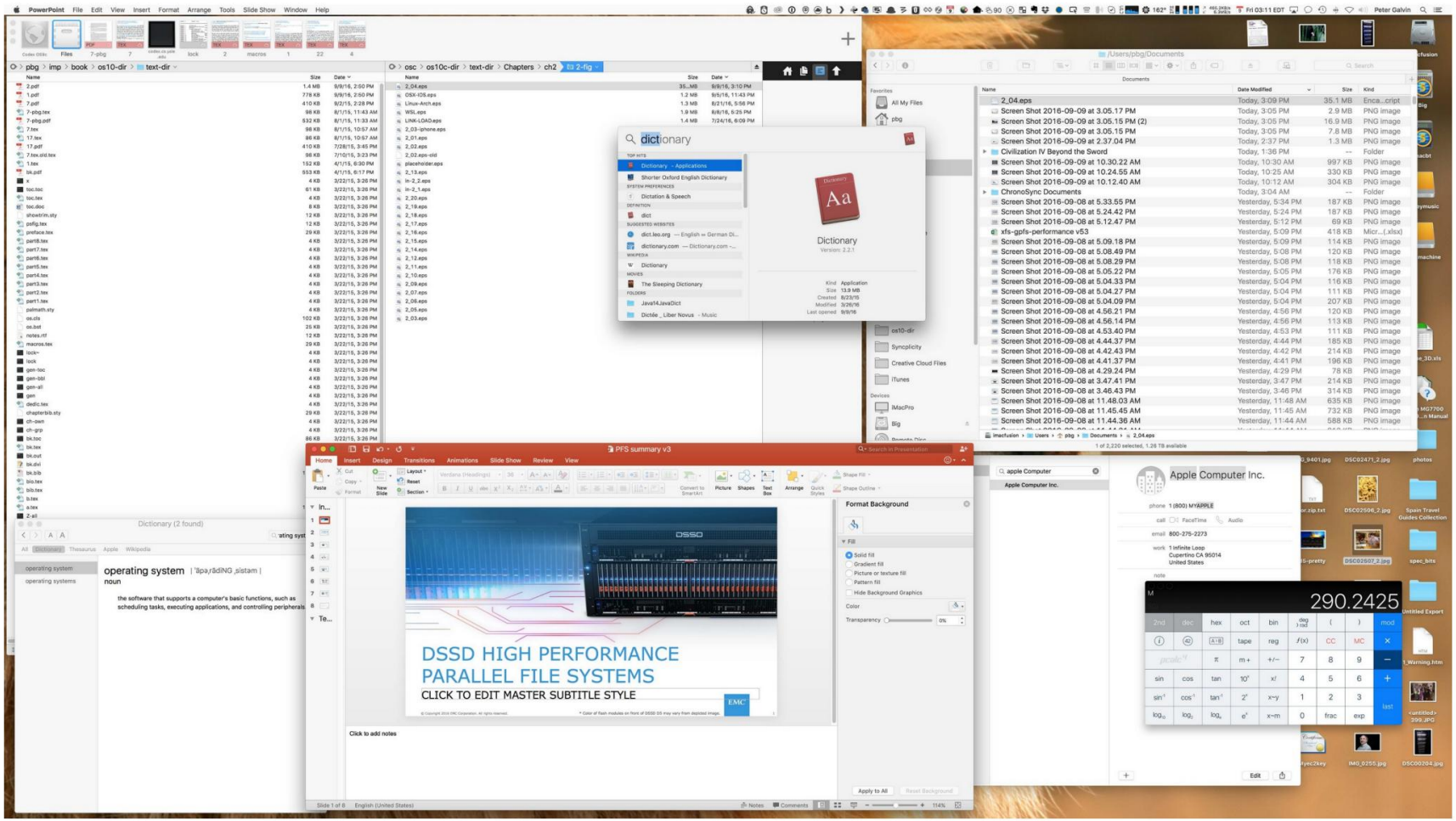


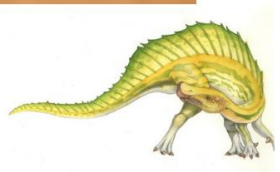


Machine Translated by Google



La interfaz gráfica de usuario de Mac OS X





Machine Translated by Google



Llamadas al sistema

Interfaz de programación para los servicios proporcionados por el sistema operativo

Generalmente se escribe en un lenguaje de alto nivel (C o C++).

Se accede principalmente mediante programas a través de una aplicación de alto nivel.

Interfaz de programación de aplicaciones (API) en lugar de utilizar llamadas directas al sistema.

Las tres API más comunes son la API Win32 para Windows, la API POSIX para

Sistemas basados en POSIX (incluidas prácticamente todas las versiones de UNIX, Linux y Mac OS X) y la API de Java para la máquina virtual de Java (JVM).

Tenga en cuenta que los nombres de las llamadas al sistema utilizados a lo largo de este texto son genéricos.

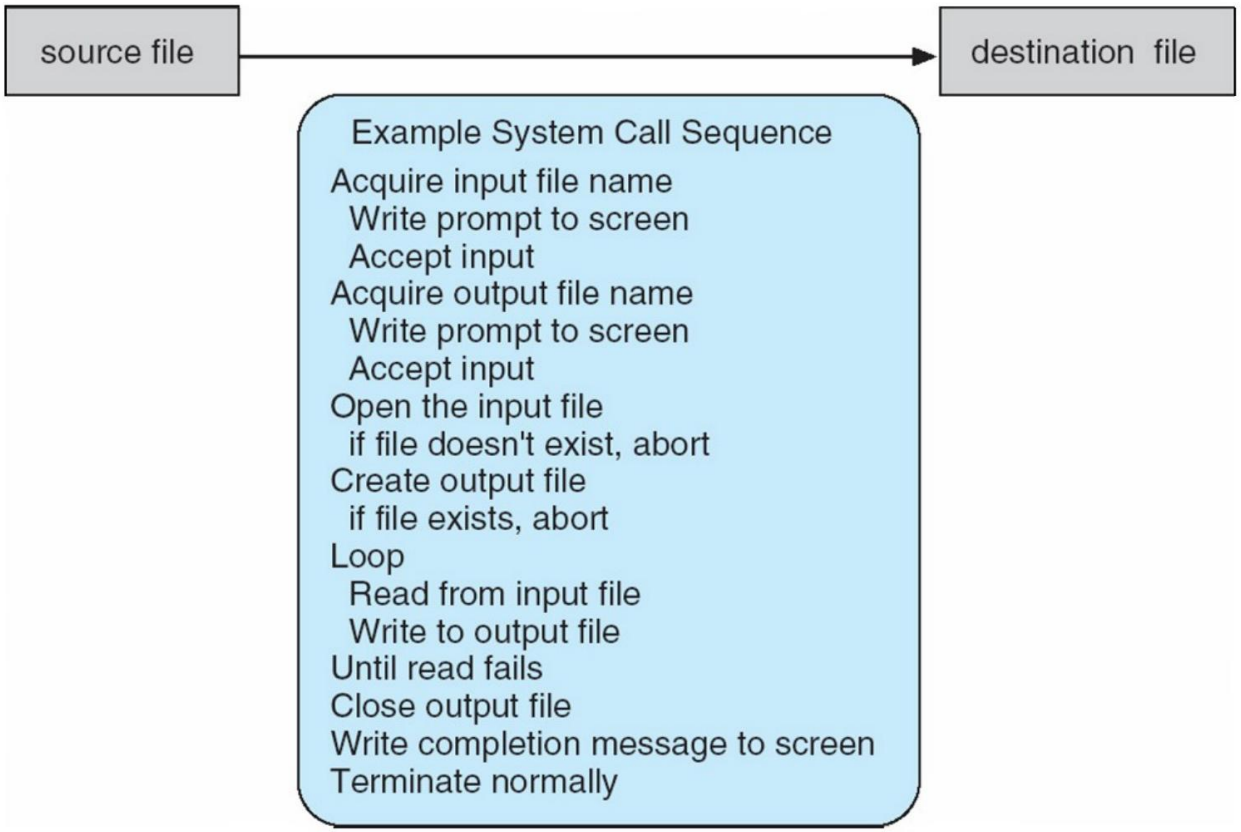


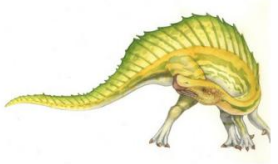
Machine Translated by Google



Ejemplo de llamadas al sistema

Secuencia de llamadas al sistema para copiar el contenido de un archivo a otro.





Machine Translated by Google



Ejemplo de API estándar

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

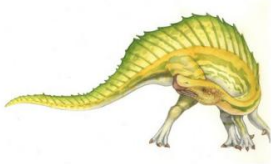
ssize_t    read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.



Machine Translated by Google



Implementación de llamadas al sistema

Normalmente, a cada llamada al sistema se le asocia un número.

- La interfaz de llamada al sistema mantiene una tabla indexada según estos números

La interfaz de llamada al sistema invoca la llamada al sistema prevista en el núcleo del sistema operativo y devuelve el estado de la llamada al sistema y cualquier valor de retorno.

El emisor de la llamada no necesita saber nada sobre cómo se implementa la llamada al sistema.

- Solo necesita obedecer la API y entender lo que hará el sistema operativo como tal.

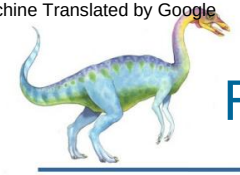
  llamada de resultado

- La mayoría de los detalles de la interfaz del sistema operativo están ocultos al programador por la API.

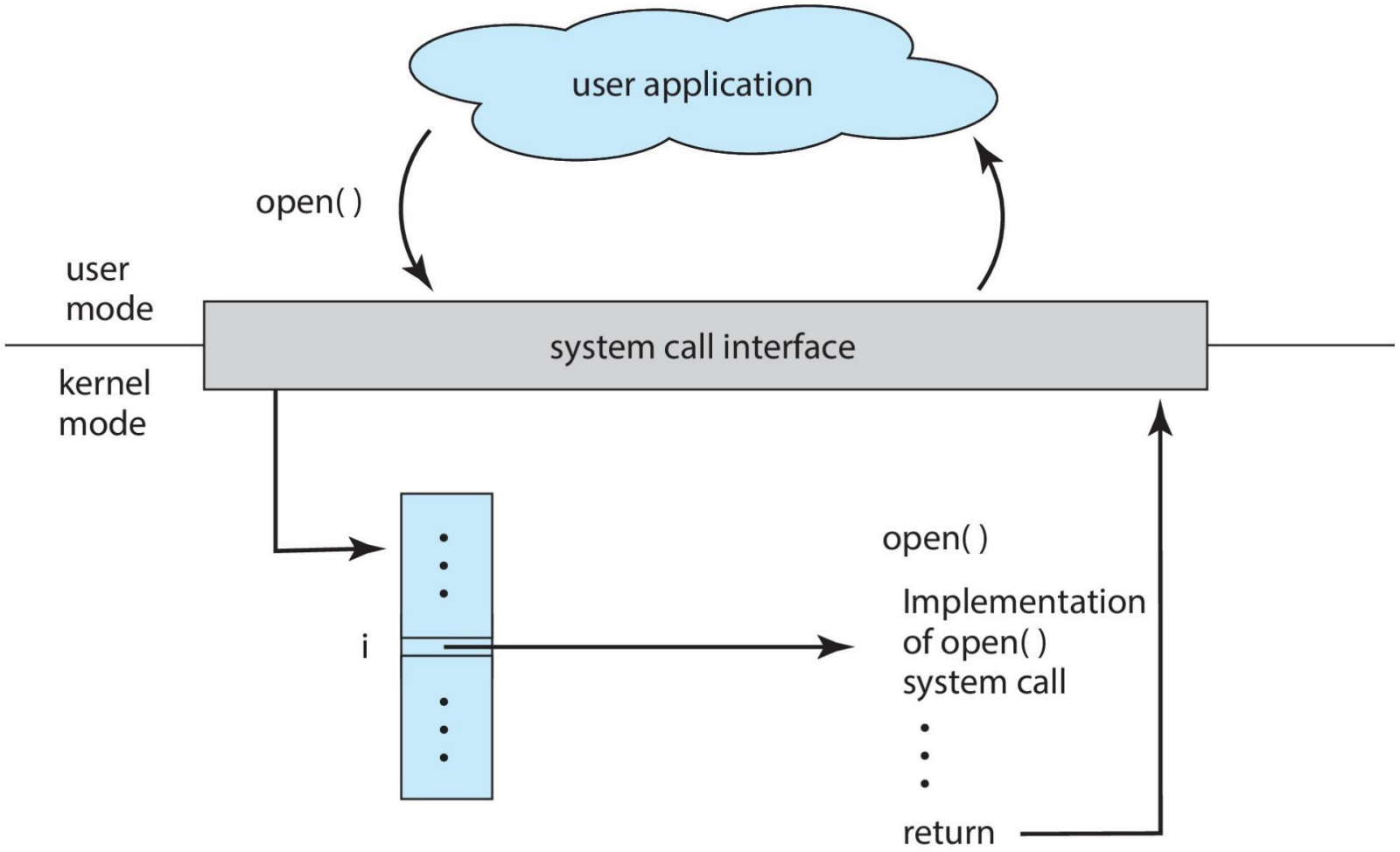
  Gestionado por la biblioteca de soporte en tiempo de ejecución (conjunto de funciones integradas en las bibliotecas incluidas con el compilador).

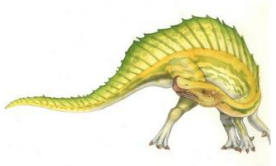


Machine Translated by Google



Relación entre API, llamadas al sistema y sistema operativo.





Machine Translated by Google



Paso de parámetros en llamadas al sistema

A menudo, se requiere más información que la simple identidad de la persona deseada.

llamada al sistema

- El tipo y la cantidad exactos de información varían según el sistema operativo y llamar

Tres métodos generales utilizados para pasar parámetros al sistema operativo

- La opción más sencilla: pasar los parámetros en registros.

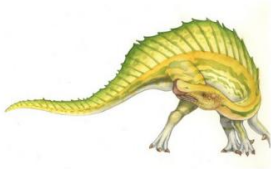
  En algunos casos, puede haber más parámetros que registros.

- Los parámetros se almacenan en un bloque o tabla en la memoria, y la dirección del bloque se pasa como parámetro en un registro.

  Este enfoque fue adoptado por Linux y Solaris.

- Parámetros colocados o insertados en la pila por el programa y eliminado de la pila por el sistema operativo

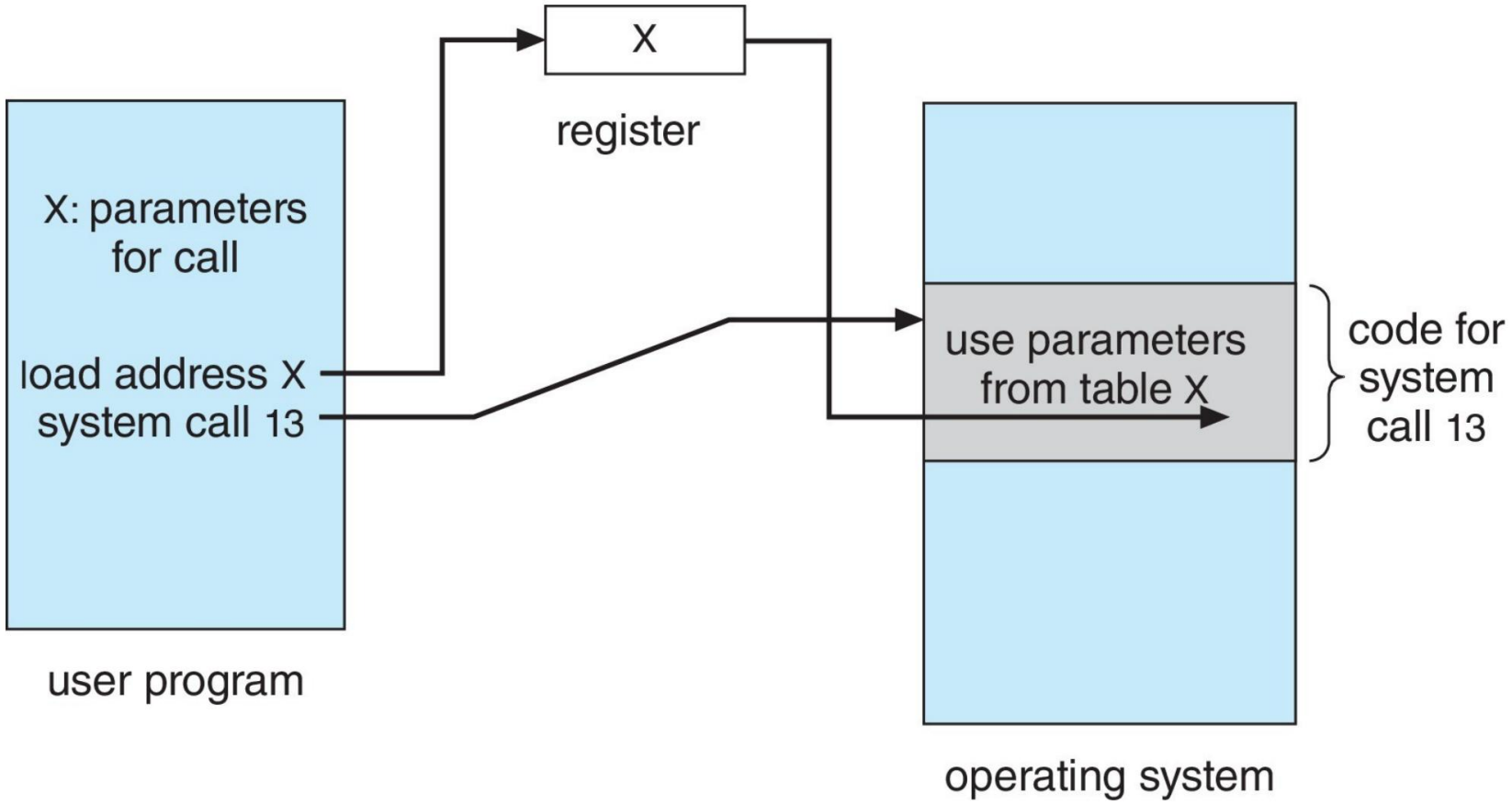
- Los métodos de bloque y pila no limitan el número ni la longitud de los parámetros que se pasan.

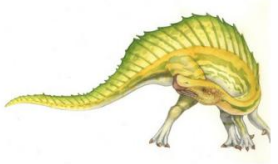


Machine Translated by Google



Paso de parámetros a través de una tabla





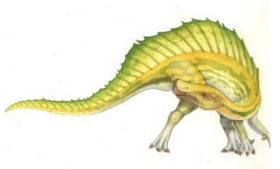
Machine Translated by Google



Tipos de llamadas al sistema

Control de procesos

- crear proceso, terminar proceso • finalizar,
  abortar • cargar,
  ejecutar • obtener
  atributos del proceso, establecer atributos del proceso
- esperar tiempo
- Evento de espera, evento
  de señal • Asignar y liberar
  memoria • Volcar memoria
  en caso de error • Depurador para detectar errores, ejecución
  paso a paso • Bloqueos para gestionar el acceso a datos compartidos entre procesos



Machine Translated by Google



Tipos de llamadas al sistema (continuación)

Gestión de archivos •

crear archivo, eliminar archivo

• abrir archivo, cerrar

archivo • leer, escribir,

reposicionar • obtener y establecer

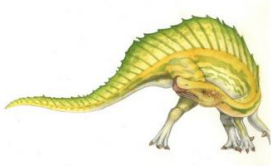
atributos de archivo Gestión

de dispositivos • solicitar dispositivo, liberar

dispositivo • leer, escribir,

reposicionar • obtener atributos de dispositivo, establecer

atributos de dispositivo • conectar o desconectar lógicamente dispositivos



Machine Translated by Google



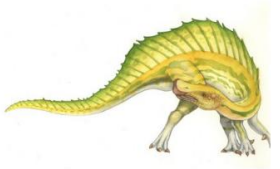
Tipos de llamadas al sistema (continuación)

Mantenimiento de la información

- Obtener la hora o la fecha, establecer la hora o la fecha
- obtener datos del sistema, establecer datos del sistema
- Obtener y establecer atributos de proceso, archivo o dispositivo

Comunicaciones

- crear, eliminar conexión de comunicación
- Enviar y recibir mensajes si el modelo de paso de mensajes se utiliza para el host.
  nombre o nombre del proceso
  Del cliente al servidor
- El modelo de memoria compartida crea y obtiene acceso a la memoria.
  regiones
- Información sobre el estado de la transferencia
- Conectar y desconectar dispositivos remotos



Machine Translated by Google



Tipos de llamadas al sistema (continuación)

Protección

- Controlar el acceso a los recursos
- Obtener y configurar permisos
- Permitir y denegar el acceso del usuario



Machine Translated by Google

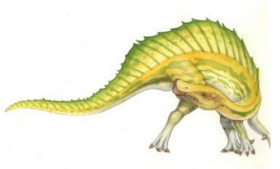


Ejemplos de llamadas al sistema de Windows y Unix

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`



Machine Translated by Google

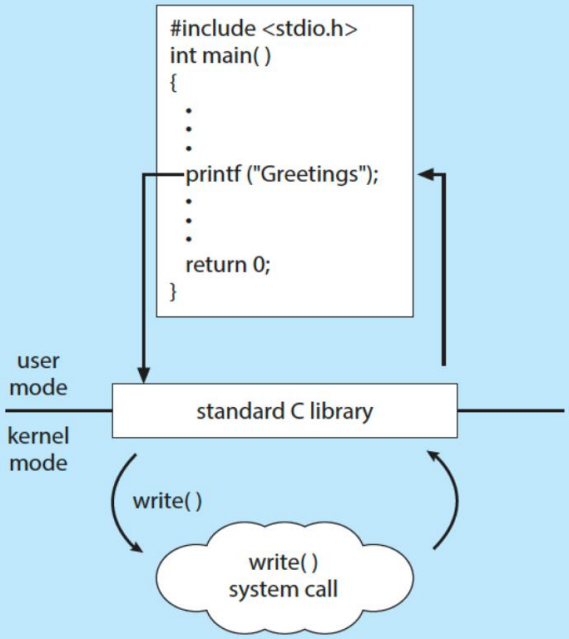


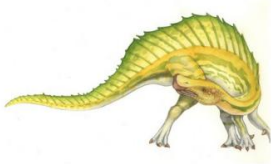
Ejemplo de biblioteca C estándar

Programa en C que invoca la llamada a la biblioteca printf(), la cual llama a la función del sistema write().

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:





Machine Translated by Google



Ejemplo: Arduino

Realizar una sola tarea

Sin sistema operativo

Programas (sketch) cargados a través de
USB a memoria flash

Espacio de memoria único

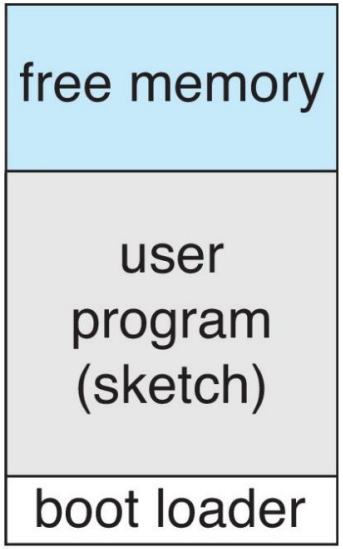
El cargador de arranque carga el programa

Salida del programa ->
recarga del shell



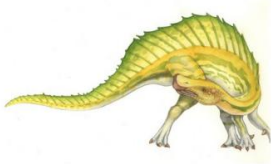
(a)

Al iniciar el sistema



(b)

ejecutar un programa



Machine Translated by Google



Ejemplo: FreeBSD

Variante Unix

Multitarea

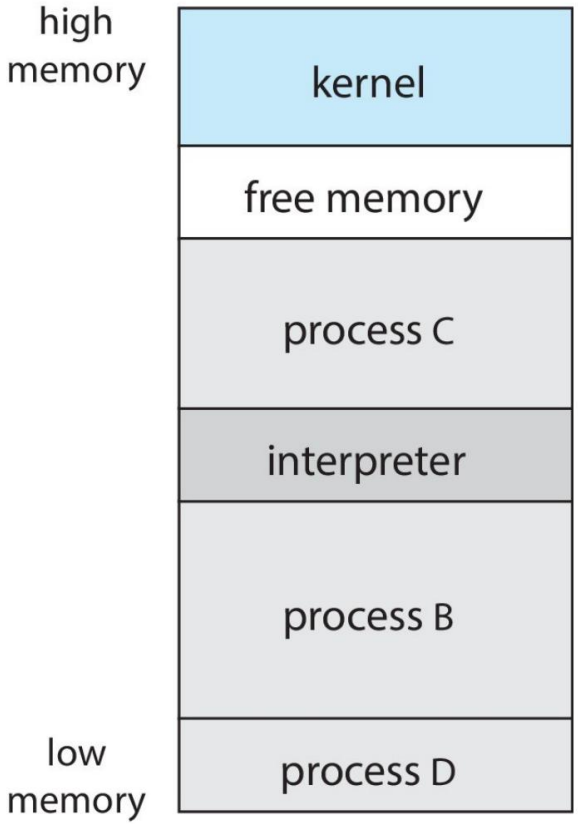
Inicio de sesión del usuario -> invocar el intérprete de comandos elegido por el usuario

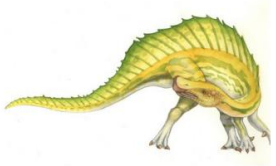
El intérprete de comandos ejecuta la llamada al sistema fork() para crear un proceso.

- Ejecuta exec() para cargar el programa en el proceso.
- El intérprete de comandos espera a que el proceso finalice o continúa con los comandos del usuario.

El proceso finaliza con:

- código = 0 – sin error
- código > 0 – código de error





Machine Translated by Google



Servicios del sistema

Los programas del sistema proporcionan un entorno conveniente para el desarrollo y la ejecución de programas. Se pueden dividir en:

- Manipulación de archivos
- La información de estado a veces se almacena en un archivo.
- Compatibilidad con lenguajes de programación
- Carga y ejecución del programa
- Comunicaciones
- Servicios de antecedentes
- Programas de aplicación

La visión que la mayoría de los usuarios tiene del sistema operativo está definida por los programas del sistema, no por las llamadas al sistema propiamente dichas.



Machine Translated by Google



Servicios del sistema (continuación)

Proporcionar un entorno conveniente para el desarrollo del programa y ejecución

- Algunas de ellas son simplemente interfaces de usuario para llamadas al sistema; otras son considerablemente más complejas.

Gestión de archivos : crear, eliminar, copiar, renombrar, imprimir, volcar, listar, y, en general, manipular archivos y directorios.

Información de estado

- Algunos solicitan información al sistema: fecha, hora, cantidad de memoria disponible, espacio en disco, número de usuarios.
- Otros proporcionan información detallada sobre el rendimiento, el registro de eventos y la depuración. información
- Normalmente, estos programas formatean e imprimen la salida en el dispositivos terminales u otros dispositivos de salida
- Algunos sistemas implementan un registro , utilizado para almacenar y recuperar información de configuración



Machine Translated by Google



Servicios del sistema (continuación)

Modificación de archivos

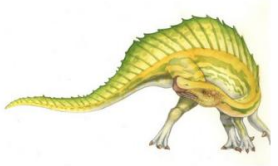
- Editores de texto para crear y modificar archivos
- Comandos especiales para buscar contenido en archivos o realizar transformaciones del texto.

Soporte para lenguajes de programación : en ocasiones se proporcionan compiladores, ensambladores, depuradores e intérpretes.

Carga y ejecución del programa: cargadores absolutos, reubicables. cargadores, editores de enlaces y cargadores superpuestos, sistemas de depuración para lenguajes de alto nivel y de máquina.

Comunicaciones : Proporcionan el mecanismo para crear conexiones virtuales entre procesos, usuarios y sistemas informáticos.

- Permitir a los usuarios enviar mensajes a las pantallas de otros usuarios, navegar por páginas web, enviar mensajes de correo electrónico, iniciar sesión de forma remota y transferir archivos de una máquina a otra.



Machine Translated by Google



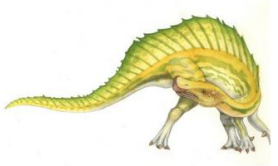
Servicios del sistema (continuación)

Servicios de antecedentes

- Iniciar al arrancar el sistema
  - Algunos para el inicio del sistema, luego terminan
  - Algunos desde el arranque del sistema hasta el apagado
- Proporcionar funciones como comprobación de disco, programación de procesos, registro de errores e impresión.
- Ejecutar en contexto de usuario, no en contexto de kernel.
- Conocidos como servicios, subsistemas, demonios

Programas de solicitud

- No pertenecen al sistema
- Gestionado por usuarios
- Normalmente no se considera parte del sistema operativo.
- Se ejecuta mediante línea de comandos, clic del ratón o toque con el dedo.



Machine Translated by Google



Enlazadores y cargadores

Código fuente compilado en archivos objeto diseñados para cargarse en cualquier ubicación de memoria física: archivo objeto reubicable .

El enlazador combina estos archivos en un único archivo ejecutable binario.

- También incluye bibliotecas

El programa reside en el almacenamiento secundario como un ejecutable binario.

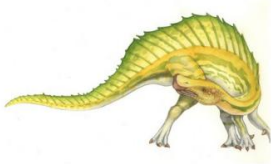
Debe ser cargado en memoria por el cargador para ser ejecutado.

- La reubicación asigna direcciones finales a las partes del programa y ajusta el código y los datos del programa para que coincidan con esas direcciones.

Los sistemas modernos de propósito general no enlazan las bibliotecas con los ejecutables.

- En cambio, las bibliotecas vinculadas dinámicamente (en Windows, DLL) son
  Se carga según sea necesario y es compartida por todos los que usan la misma versión de esa misma biblioteca (se carga una sola vez).

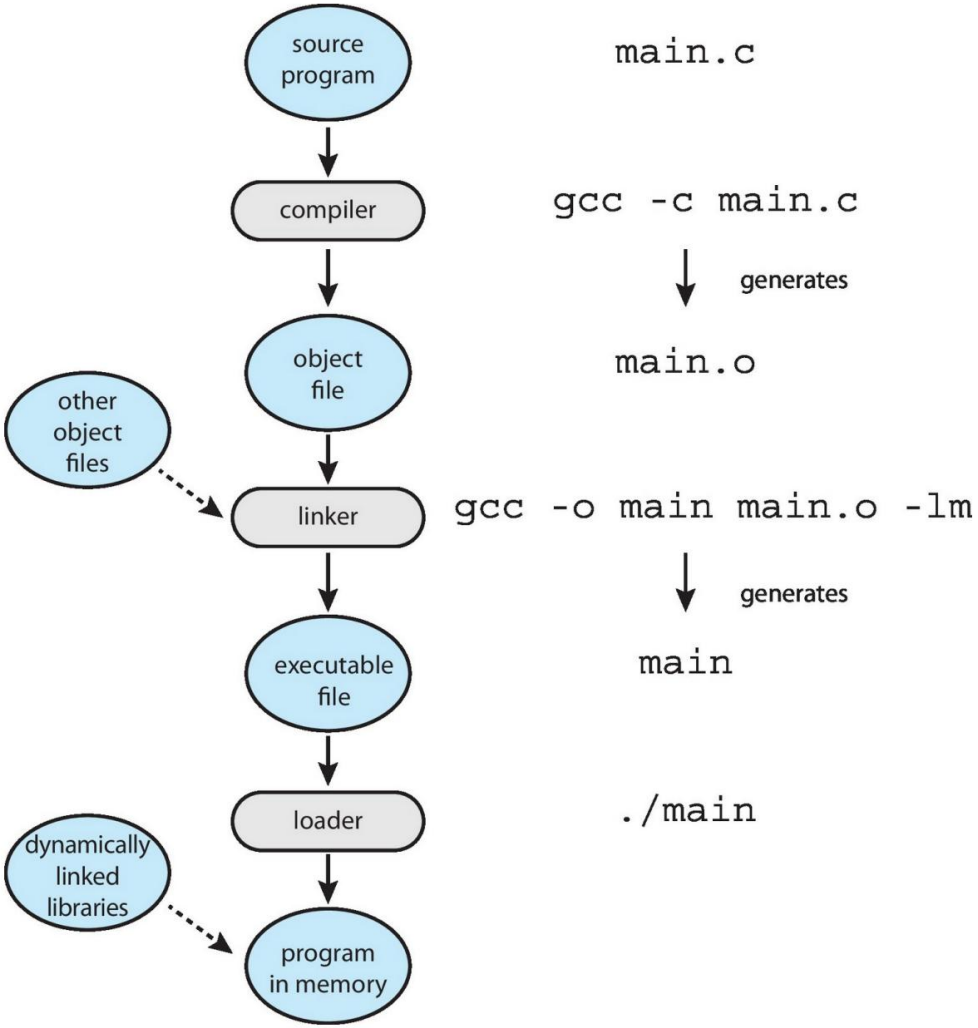
Los archivos ejecutables de objetos tienen formatos estándar, por lo que el sistema operativo sabe cómo cargarlos e iniciarlos

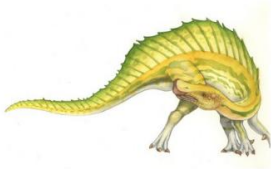


Machine Translated by Google



El papel del enlazador y el cargador





Machine Translated by Google



Por qué las aplicaciones son específicas del sistema operativo.

Las aplicaciones compiladas en un sistema generalmente no se pueden ejecutar en otros sistemas operativos.

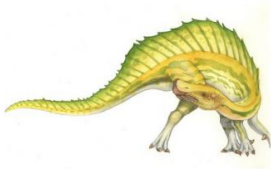
Cada sistema operativo proporciona sus propias llamadas al sistema únicas.

- Formatos de archivo propios, etc.

Las aplicaciones pueden ser multisistema operativo.

- Escrito en un lenguaje interpretado como Python, Ruby, y con intérprete disponible en múltiples sistemas operativos.
- Aplicación escrita en un lenguaje que incluye una máquina virtual que contiene la aplicación en ejecución (como Java).
- Utilice un lenguaje estándar (como C), compile por separado en cada sistema operativo para ejecutar en cada

La interfaz binaria de la aplicación (ABI) es el equivalente arquitectónico de la API, y define cómo los diferentes componentes del código binario pueden interactuar para un sistema operativo determinado en una arquitectura, CPU, etc. determinada.



Machine Translated by Google



Diseño e implementación

El diseño e implementación de sistemas operativos no es “resoluble”, pero algunos Los enfoques han demostrado ser exitosos.

La estructura interna de los diferentes sistemas operativos puede variar ampliamente.

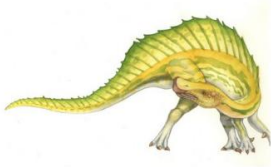
Comience el diseño definiendo los objetivos y las especificaciones .

Influye la elección del hardware y el tipo de sistema.

Objetivos del usuario y objetivos del sistema

- Objetivos del usuario: el sistema operativo debe ser cómodo de usar, fácil de aprender, fiable, seguro y rápido.
- Objetivos del sistema: el sistema operativo debe ser fácil de diseñar, implementar y mantener, además de ser flexible, confiable, libre de errores y eficiente.

Especificar y diseñar un sistema operativo es una tarea altamente creativa del software. ingeniería



Machine Translated by Google



Política y Mecanismo

Política: ¿Qué hay que hacer?

- Ejemplo: Interrumpir cada 100 segundos

Mecanismo: ¿Cómo hacer algo?

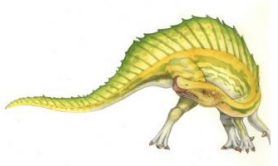
- Ejemplo: temporizador

Principio importante: separar la política del mecanismo.

La separación de la política del mecanismo es muy

Es un principio importante que permite la máxima flexibilidad en caso de que sea necesario modificar las decisiones políticas posteriormente.

- Ejemplo: cambiar 100 a 200



Machine Translated by Google



Implementación

Mucha variación

- Primeros sistemas operativos en lenguaje ensamblador
- Luego, lenguajes de programación de sistemas como Algol, PL/1
- Ahora C, C++

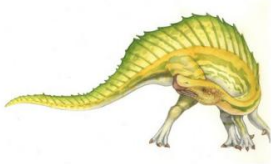
En realidad, suele ser una mezcla de idiomas.

- Niveles más bajos en el ensamblaje
- Cuerpo principal en C
- Programas de sistemas en C, C++, lenguajes de scripting como PERL, Python, scripts de shell

Un lenguaje de alto nivel es más fácil de portar a otro hardware.

- Pero más lento

La emulación permite que un sistema operativo se ejecute en hardware no nativo.



Machine Translated by Google



Estructura del sistema operativo

El sistema operativo de propósito general es un programa muy grande.

Diversas formas de estructurar las propias

- Estructura simple – MS-DOS
- Más complejo – UNIX
- En capas: una abstracción
- Microkernel – Mach



Machine Translated by Google



Estructura monolítica – UNIX original

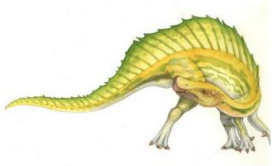
UNIX – limitado por la funcionalidad del hardware, el sistema operativo UNIX original tenía una estructura limitada.

El sistema operativo UNIX consta de dos partes separables.

- Programas de sistemas
- El núcleo

  Consta de todo lo que se encuentra por debajo de la interfaz de llamadas al sistema y por encima del hardware físico.

  Proporciona el sistema de archivos, la planificación de la CPU y la memoria. administración y otras funciones del sistema operativo; una gran cantidad de funciones para un nivel

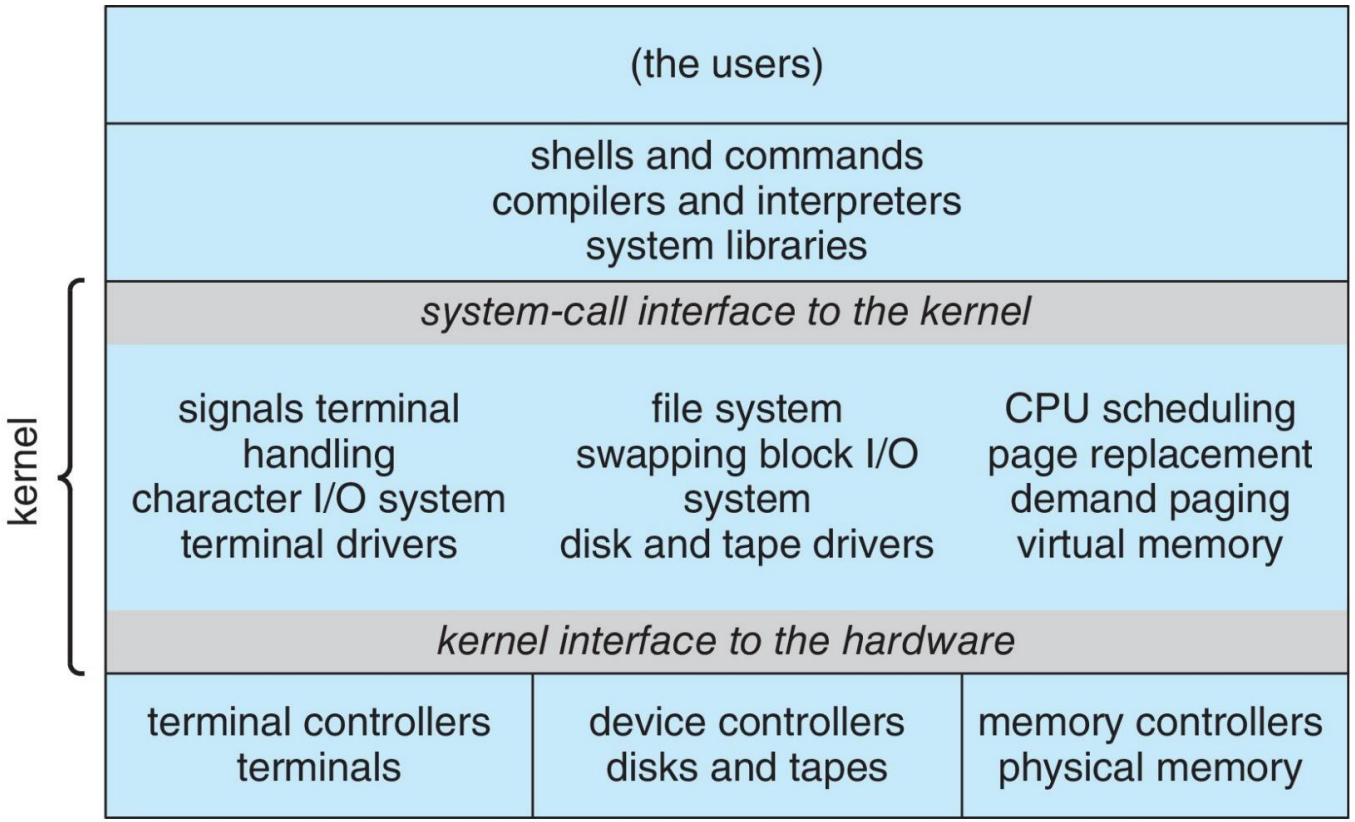


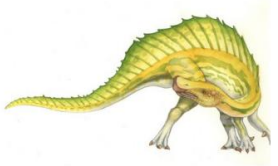
Machine Translated by Google



Estructura tradicional del sistema UNIX

Más allá de lo simple, pero no completamente estratificado.



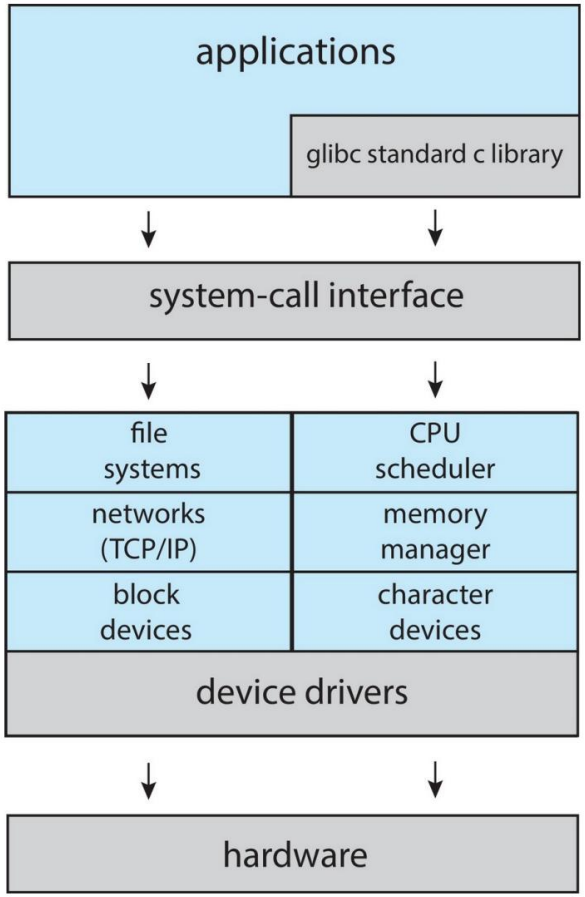


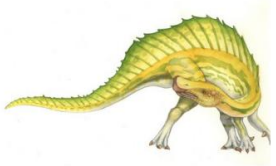
Machine Translated by Google



Estructura del sistema Linux

Diseño monolítico y modular





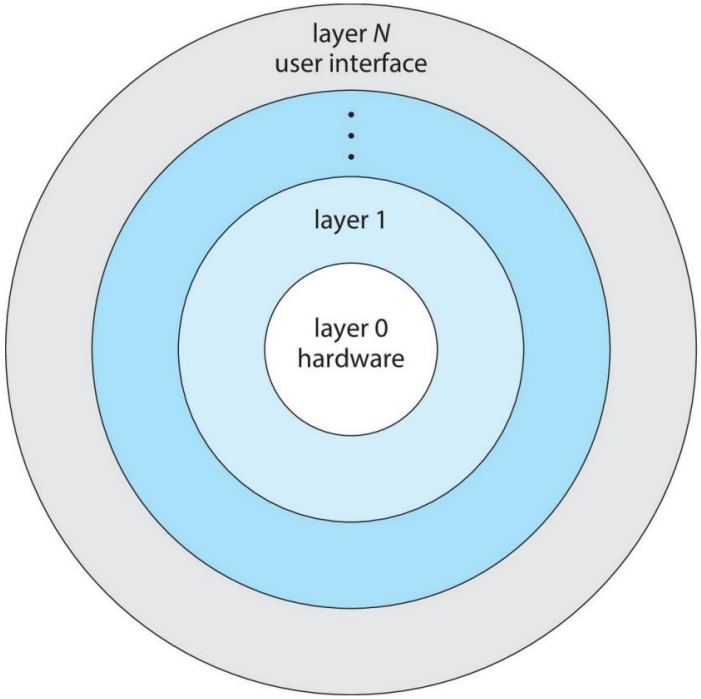
Machine Translated by Google



Enfoque por capas

El sistema operativo se divide en varias capas (niveles), cada una construida sobre capas inferiores. La capa inferior (capa 0) es el hardware; la superior (capa N) es la interfaz de usuario.

Con la modularidad, las capas son seleccionados de tal manera que cada uno utilice funciones (operaciones) y servicios de capas de nivel inferior únicamente.





Machine Translated by Google



Micronúcleos

Traslada la mayor cantidad posible de tareas del núcleo al espacio de usuario.

Mach es un ejemplo de microkernel

- El núcleo de Mac OS X (Darwin) está parcialmente basado en Mach

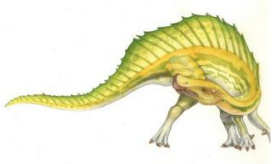
La comunicación se produce entre los módulos de usuario mediante paso de mensajes

Beneficios:

- Es más fácil extender un microkernel
- Facilita la adaptación del sistema operativo a nuevas arquitecturas.
- Más fiable (se ejecuta menos código en modo kernel)
- Más seguro

Perjuicios:

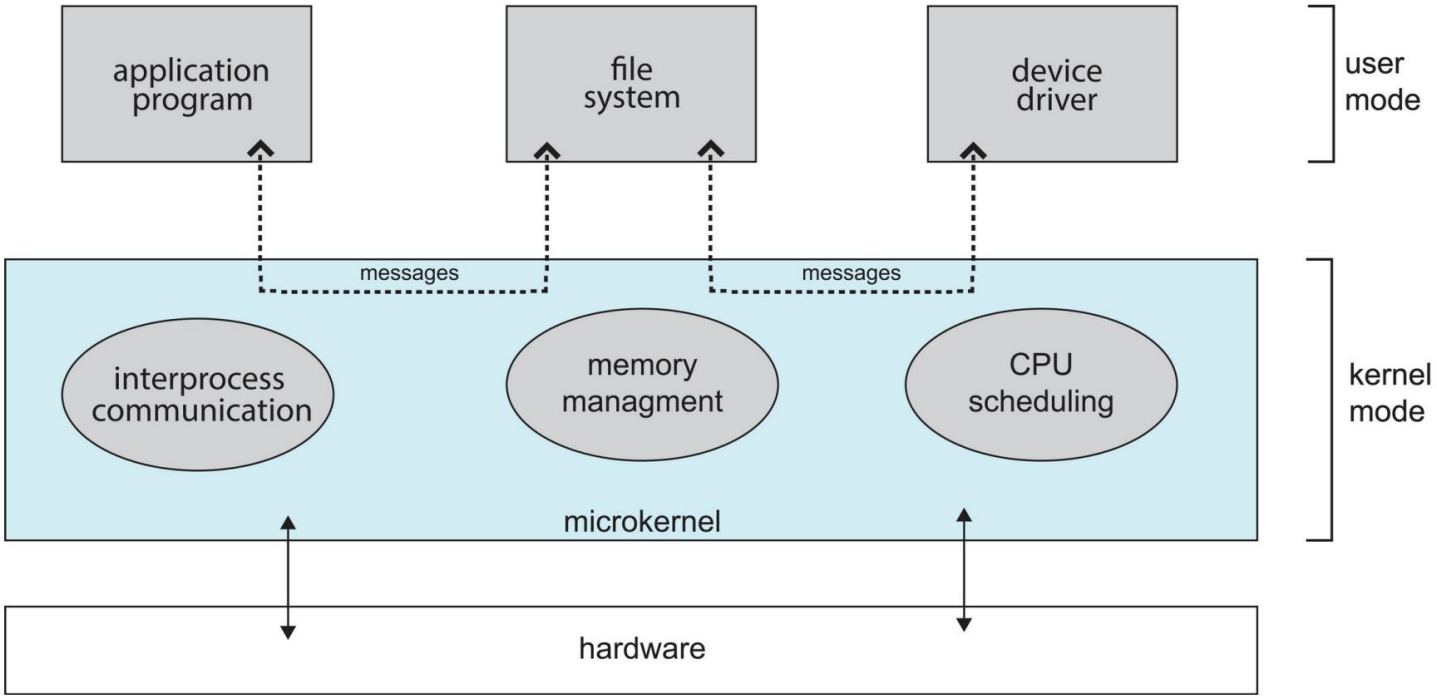
- Sobrecarga de rendimiento de la comunicación entre el espacio de usuario y el espacio del kernel

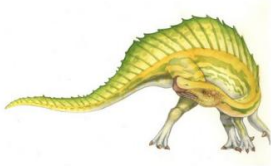


Machine Translated by Google



Estructura del sistema de micronúcleos





Machine Translated by Google



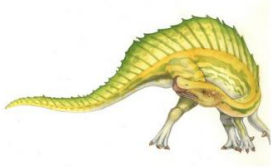
Módulos

Muchos sistemas operativos modernos implementan un núcleo cargable. módulos (LKM)

- Utiliza un enfoque orientado a objetos.
- Cada componente principal es independiente
- Cada uno se comunica con los demás a través de interfaces conocidas.
- Cada uno se puede cargar según sea necesario dentro del kernel.

En general, similar a las capas pero con mayor flexibilidad.

- Linux, Solaris, etc.



Machine Translated by Google



Sistemas híbridos

La mayoría de los sistemas operativos modernos no son un modelo puro.

- El enfoque híbrido combina múltiples métodos para abordar el rendimiento, necesidades de seguridad y usabilidad
- Los núcleos de Linux y Solaris se encuentran en el espacio de direcciones del núcleo, por lo que son monolíticos, además de modulares para la carga dinámica de funcionalidades.
- Windows es mayoritariamente monolítico, además de contar con un microkernel para diferentes subsistemas .

Apple Mac OS X híbrido, por capas, Aqua UI más programación Cocoa ambiente

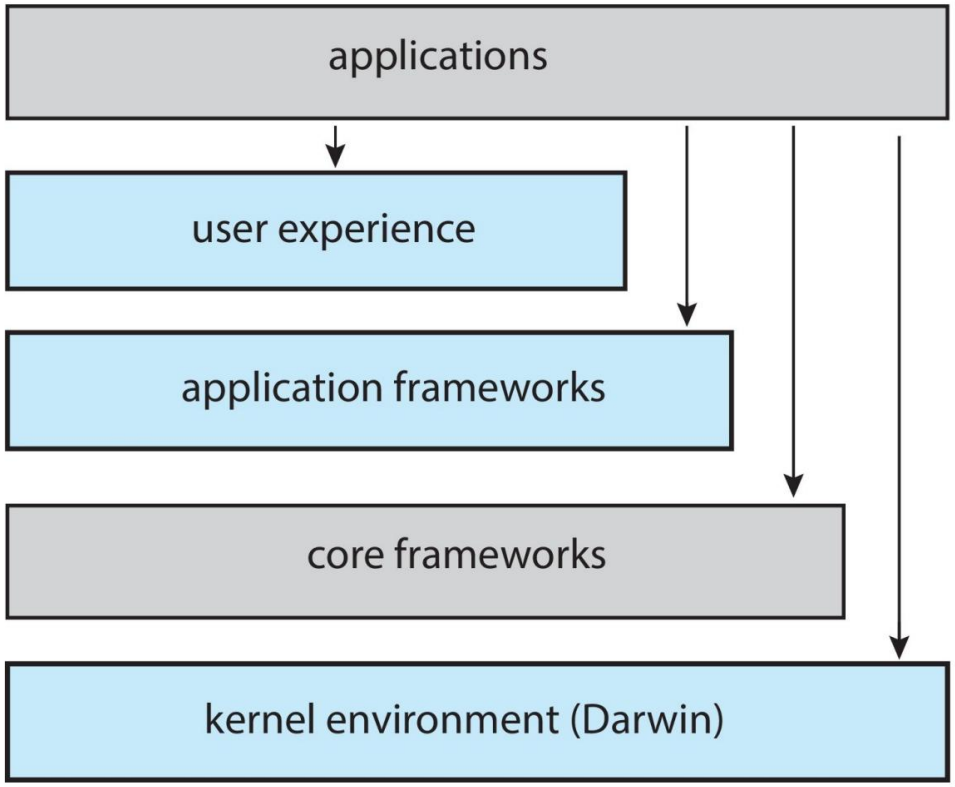
- A continuación se muestra el núcleo, que consta del micronúcleo Mach y partes de BSD Unix, además del kit de E/S y módulos de carga dinámica (llamados extensiones del núcleo).

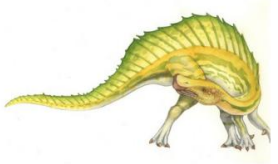


Machine Translated by Google



Estructura de macOS e iOS

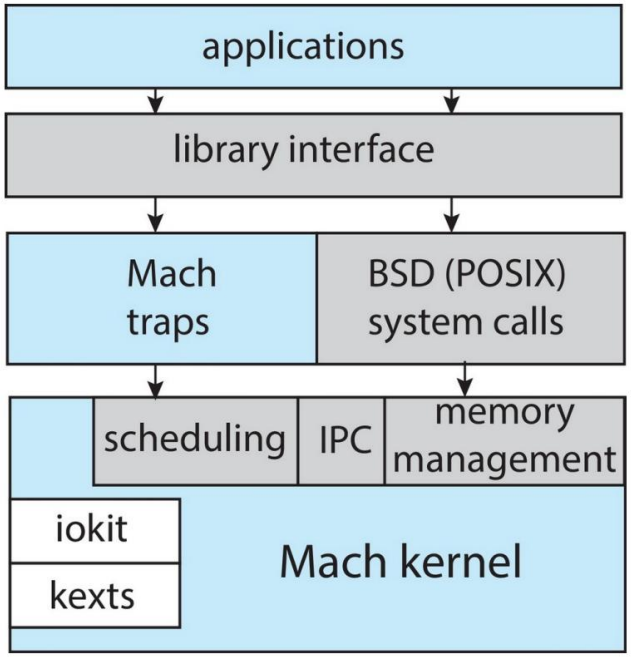


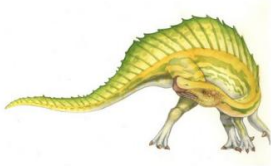


Machine Translated by Google



Darwin





Machine Translated by Google



Androide

Desarrollado por Open Handset Alliance (principalmente Google)

- Código abierto

Pila similar a la de iOS

Basado en el kernel de Linux pero modificado

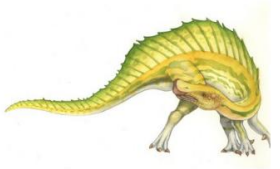
- Proporciona gestión de procesos, memoria y controladores de dispositivos.
- Añade gestión de energía

El entorno de ejecución incluye un conjunto básico de bibliotecas y Dalvik.
máquina virtual

- Aplicaciones desarrolladas en Java más la API de Android.

  Los archivos de clase Java se compilan a bytecode Java y luego se traducen a ejecutables que se ejecutan en la máquina virtual Dalvik.

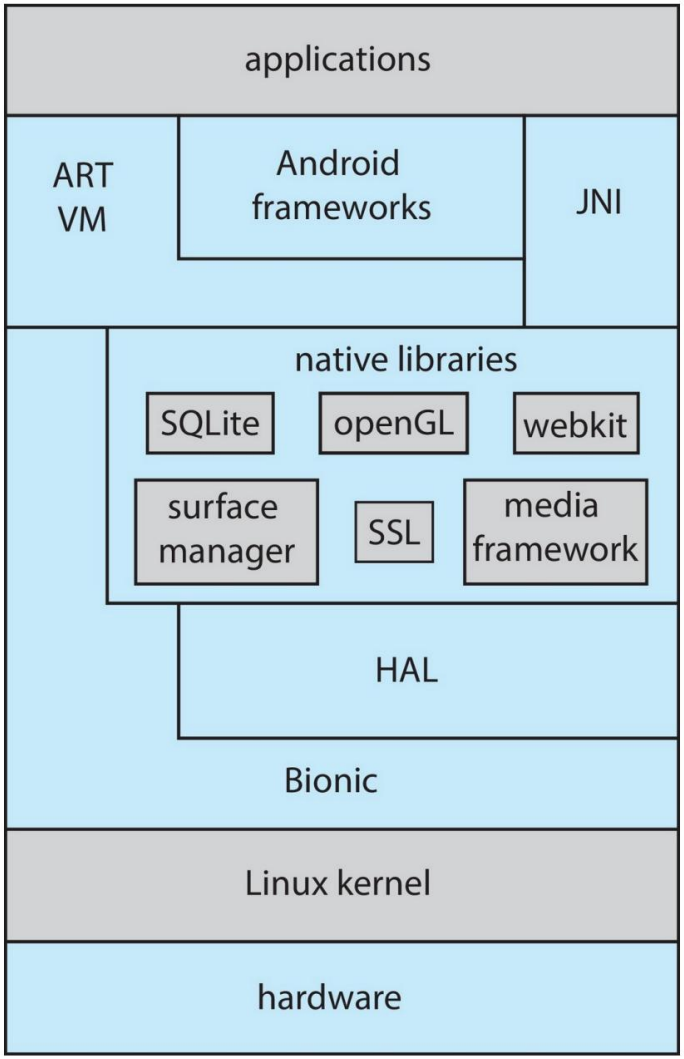
Las bibliotecas incluyen marcos de trabajo para navegadores web (webkit), bases de datos (SQLite), multimedia, libc más pequeño



Machine Translated by Google



Arquitectura de Android





Machine Translated by Google



Creación e inicio de un sistema operativo

Los sistemas operativos generalmente están diseñados para ejecutarse en una clase de sistemas con una variedad de periféricos.

Comúnmente, el sistema operativo ya está instalado en el dispositivo comprado. computadora

- Pero puede construir e instalar otros sistemas operativos.
- Si se genera un sistema operativo desde cero

  Escribir el código fuente del sistema operativo

  Configure el sistema operativo para el sistema en el que se instalará. correr

  Compilar el sistema operativo

  Instalar el sistema operativo

  Arranca el ordenador y su nuevo sistema operativo.



Machine Translated by Google



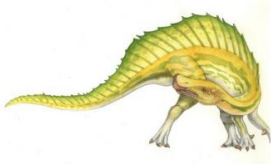
Compilación e inicio de Linux

Descarga el código fuente de Linux (http://www.kernel.org)

Configura el kernel mediante “make menuconfig”.

Compilar el kernel usando “make”

- Produce vmlinuz, la imagen del kernel
- Compilar módulos del kernel mediante “make modules”
- Instale los módulos del kernel en vmlinuz mediante “make modules_install”.
- Instale el nuevo kernel en el sistema mediante “make install”.



Machine Translated by Google



Arranque del sistema

Cuando se inicia la alimentación del sistema, la ejecución comienza en una memoria fija.
ubicación

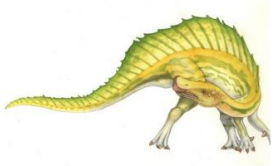
El sistema operativo debe estar disponible para el hardware para que el hardware
puede iniciarlo

- Pequeño fragmento de código: cargador de arranque, BIOS, almacenado en ROM o
  La EEPROM localiza el núcleo, lo carga en la memoria y lo inicia.
- A veces, se trata de un proceso de dos pasos en el que el bloque de arranque en una ubicación fija
  se carga mediante código ROM, que a su vez carga el cargador de arranque desde el disco.
- Los sistemas modernos reemplazan la BIOS con un sistema extensible unificado.
  Interfaz de firmware (UEFI)

El cargador de arranque común, GRUB, permite la selección del kernel desde
Múltiples discos, versiones, opciones del kernel

El núcleo se carga y el sistema se pone en marcha.

Los cargadores de arranque frecuentemente permiten varios estados de arranque, como el de usuario único.
modo



Machine Translated by Google



Depuración del sistema operativo

La depuración consiste en encontrar y corregir errores o fallos .

También ajuste del rendimiento

El sistema operativo genera archivos de registro que contienen información de error.

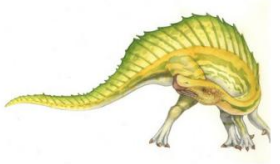
El fallo de una aplicación puede generar la captura de un archivo de volcado de memoria .
memoria del proceso

Un fallo del sistema operativo puede generar un archivo de volcado de memoria que contiene
el núcleo.

Más allá de los fallos, la optimización del rendimiento puede mejorar el rendimiento del sistema.

- A veces se utilizan listados de seguimiento de actividades, registrados para su análisis.
- El perfilado consiste en el muestreo periódico de punteros de instrucciones para buscar tendencias estadísticas.

Ley de Kernighan: “Depurar es el doble de difícil que escribir el código en primer lugar. Por lo tanto, si escribes el código de la forma más ingeniosa posible, por definición, no eres lo suficientemente inteligente como para depurarlo”.



Machine Translated by Google

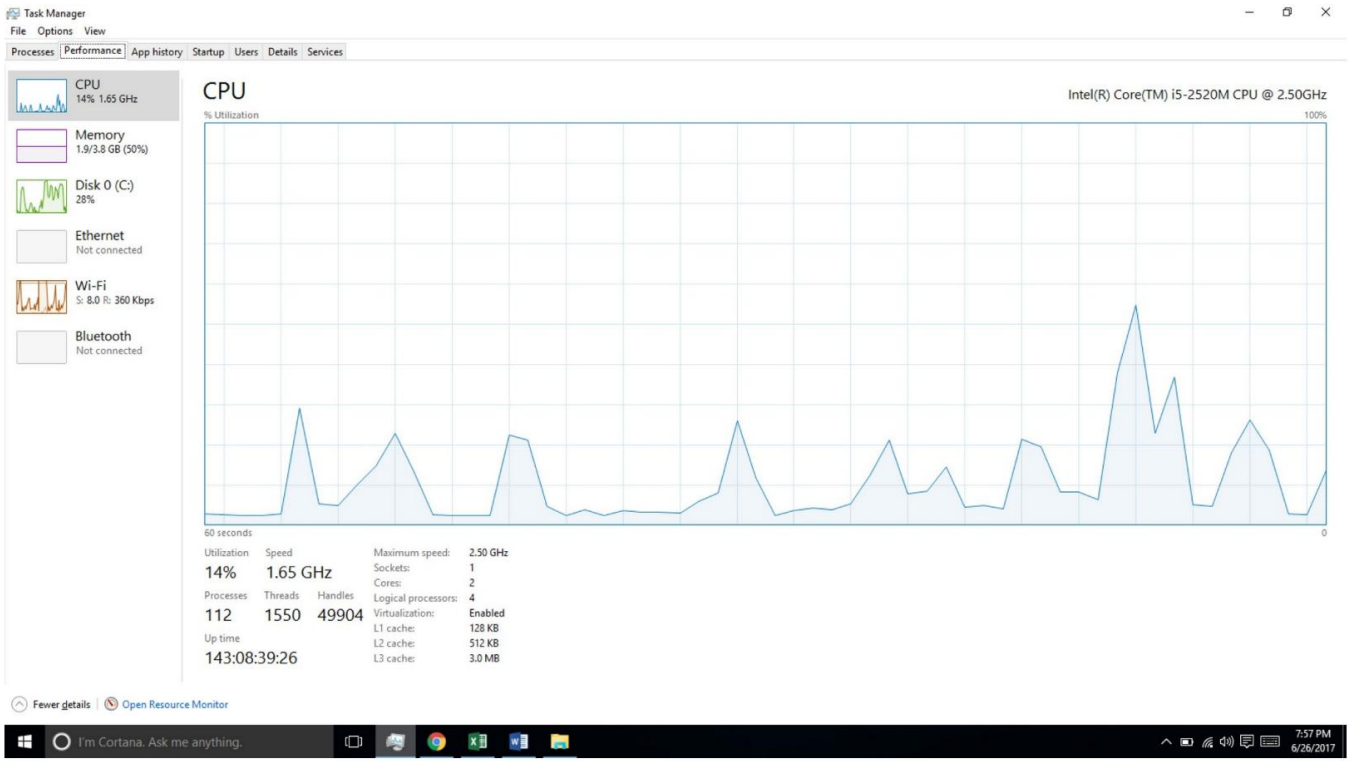


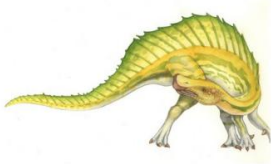
Ajuste del rendimiento

Mejorar el rendimiento eliminando los cuellos de botella.

El sistema operativo debe proporcionar medios para calcular y mostrar medidas de comportamiento del sistema

Por ejemplo, el programa “top” o el Administrador de tareas de Windows.





Machine Translated by Google

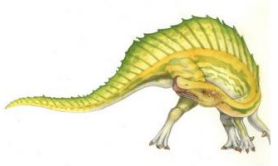


Rastreo

Recopila datos para un evento específico, como los pasos involucrados en la invocación de una llamada al sistema.

Las herramientas incluyen:

- strace: rastrea las llamadas al sistema invocadas por un proceso
- gdb – depurador a nivel de código fuente
- perf – conjunto de herramientas de rendimiento de Linux
- tcpdump: recopila paquetes de red



Machine Translated by Google



BCC

Depurar las interacciones entre el código de nivel de usuario y el código del kernel es prácticamente imposible sin un conjunto de herramientas que comprenda ambos y pueda instrumentar sus acciones.

BCC (BPF Compiler Collection) es un completo conjunto de herramientas que proporciona funciones de rastreo para Linux.

- Véase también el DTrace original

Por ejemplo, disksnoop.py rastrea la actividad de E/S del disco.

```
TIME(s)          T      BYTES      LAT(ms)
1946.29186700    R      8              0.27
1946.33965000    R      8              0.26
1948.34585000    W      8192           0.96
1950.43251000    R      4096           0.56
1951.74121000    R      4096           0.35
```

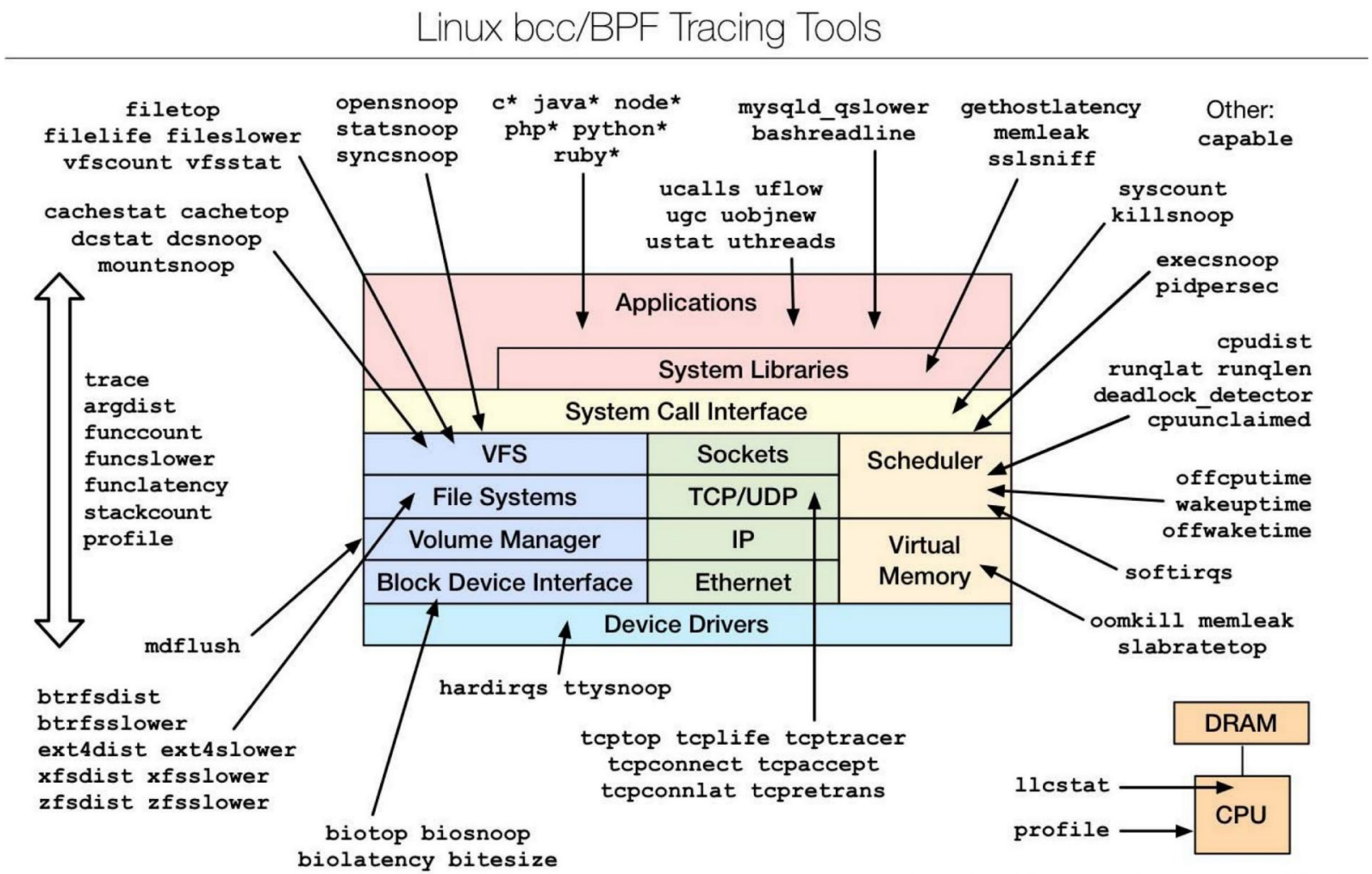
Muchas otras herramientas (siguiente diapositiva)

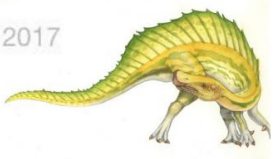


Machine Translated by Google



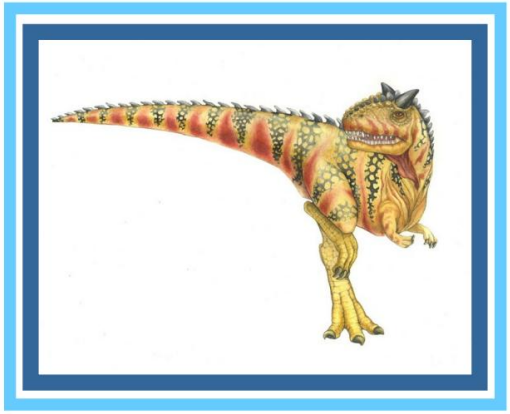
Herramientas de rastreo bcc/BPF para Linux





Machine Translated by Google

Fin del capítulo 2



Silberschatz, Galvin y Gagne ©2018